

2026-04-29

HydraR: Agentic Infrastructure for Reproducible Scientific Research

Chi Nam (Ignatius) Pang, Aidan Tay

Executive Summary

Large Language Models (LLMs) are catalyzing a paradigm shift in scientific computing, fundamentally transforming how research is performed. This transformation is particularly evident in the field of bioinformatics, where LLMs have evolved beyond simple chatbots into autonomous “agents” that can plan and execute complex tasks that previously demanded computational expertise, and powerful multi-agent systems, where a team of AI agents can autonomously collaborate with one another to tackle multi-faceted problems. However, the lack of a robust framework to support the development and use of agentic systems in the R ecosystem poses a significant risk, whereby the ephemeral nature of agent reasoning and potential destructive filesystem modifications, threaten to undermine the integrity of R-based research. Because of this, there is a critical need for traceable, stateful and secure agent orchestration. In this project, we address this need by building on HydraR, an existing R package dedicated to the scientific reproducibility of agentic systems. Specifically, this project has two distinct objectives. First, we aim to develop a comprehensive memory and state management system for automated retries of failed tasks, execution caching and state retrieval for skipping duplicated tasks and resource optimization, or agent validation for workflow resilience. Second, we aim to develop an environment management system that integrates with filesystem and runtime isolation tools such as Git worktree and Docker to ensure that agents can iterate and fail safely without risking the integrity of sensitive research environments. Thus, by serving as the central hub for traceable, stateful and secure agentic orchestration, HydraR will empower computational biologists and the broader R community to adopt agentic systems with confidence, and build sophisticated, transparent and reproducible AI-driven workflows that uphold the rigor and integrity of R-based scientific research.

Budget

We are requesting **\$14,400 USD** in funding for expert software developers to work on this project over a 7-week period (approximately 245 hours).

Signatories

Project team

- **Chi Nam (Ignatius) Pang - Co-Primary Investigator.** Senior Bioinformatician at Macquarie University / Australian Proteome Analysis Facility (APAF). Expert in R package development, proteomics pipelines, and reproducibility standards.
- **Aidan Tay - Co-Primary Investigator.** Bioinformatician at APAF. Architect of the HydraR R6 DAG engine, Git Worktree isolation logic, and the custom state-hashing persistence Layer.

Contributors

- **APAF Bioinformatics Team:** Provided the testing ground for initial prototypes in high-throughput proteomics and metabolomics research.
- **Antigravity AI:** Agentic assistant used during the rapid prototyping and documentation hardening phase of the project (as disclosed in `agents.md`).

Community Supporters

- **Hanna Szafranska:** PhD student at University of Cambridge.
- **Nathalie Nataren:** PhD Student at Adelaide University.
- **Tyrone Chen:** Bioinformatician at Peter MacCallum Cancer Centre.
- **Calum Walsh:** Bioinformatician at University of Melbourne.
- **Johan Gustafsson:** Research Community Engagement Lead at Australian BioCommons.

Consulted

- **rOpenSci Editorial Board:** Initial discussions regarding HydraR’s role in the AI-R ecosystem.
- **Macquarie University:** Consulted on the impact of agentic orchestration for national-scale omics infrastructure.
- **Henrik Bengtsson:** Consulted on HydraR package design and package infrastructure.
- **Jeroen Ooms:** Consulted on HydraR package design and package infrastructure.

The Problem

Large Language Models (LLMs) are catalyzing a paradigm shift in scientific computing, fundamentally transforming how research is performed. This transformation is particularly evident in the field of bioinformatics, where LLMs have evolved beyond simple chatbots into autonomous “agents” that can plan and execute complex tasks previously requiring computational expertise, and into powerful multi-agent systems, where a team of AI agents can autonomously collaborate to tackle multifaceted problems. By democratizing access to these sophisticated multi-agent systems, this automation not only accelerates biological discovery but also empowers the broader scientific community to harness high-performance workflows once restricted to deep domain specialists.

Yet despite offering many great benefits, **the current R ecosystem lacks the necessary framework for traceable, stateful, and secure agent orchestration.** Existing R packages such as `ellmer` and `mall` provide excellent interfaces to LLM providers to send prompts and parse responses but fail to record the entire decision-making process of an agent, making the “journey” ephemeral and untraceable. Other packages such as `LLMAgentR` and `puppeteer` enable agent orchestration, but

lack the infrastructure for state management, making it impossible for multi-agent systems to meet the rigorous reproducibility standards of modern bioinformatics. Meanwhile, advanced frameworks such as **LangGraph** and **AutoGen** in the Python ecosystem lack the necessary features for auditable agent orchestration, rendering them unsuitable for R-based research via Python-based bridges (i.e., **reticulate**). Moreover, granting autonomous agents unchecked access to research environments can be dangerous as their actions can inadvertently lead to file modifications, deletions, or corruption of the working directory. Thus, to safeguard the integrity of R-based research, a robust framework must exist to support the development and reproducible use of agentic systems.

To address this critical gap in the R ecosystem, this project will build on HydraR (<https://github.com/APAF-bioinformatics/HydraR>), an existing R package dedicated to the scientific reproducibility of agentic systems. Specifically, we aim to develop:

1. A comprehensive memory and state management system for logging ephemeral agent reasoning and agent interactions.
2. A robust environment management system for isolating agents from the host environment.

By delivering these systems, **we will equip computational biologists with the necessary tools for developing production-grade agentic systems for bioinformatics**, such as automatic retry of failed tasks, reduction of redundant computations, and safe execution of agents without risking the integrity of sensitive research environments. Crucially, **this agentic-agnostic framework guarantees that HydraR can integrate with any existing or future agent orchestration package**, thereby transforming HydraR into an infrastructure hub for the next generation of agentic tools. By serving as the central foundation for traceable, stateful, and secure agent orchestration, HydraR will empower computational biologists and R developers alike to adopt agentic systems with confidence, enabling them to build sophisticated, reproducible AI-driven workflows that uphold the rigorous standards of scientific research.

The Proposal

Overview

This project aims to address the critical need for traceable, stateful, and secure agent orchestration in the R ecosystem. To address this need, we will build on HydraR (<https://github.com/APAF-bioinformatics/HydraR>), an existing R package dedicated to the scientific reproducibility of agentic systems. In particular, **we aim to develop a comprehensive memory and state management system for logging ephemeral agent reasoning and agent interactions, and a robust environment management system for isolating agents from the host environment.** By delivering these systems, we will equip computational biologists with the necessary tools for developing production-grade agentic systems for bioinformatics, such as automatic retry of failed tasks, reduction of redundant computations, and safe execution of agents without risking the integrity of sensitive research environments. Crucially, **HydraR will provide an agent-agnostic framework that can integrate with any existing or future agent orchestration package, thereby transforming HydraR into an infrastructure hub for the next generation of agentic tools.**

Details

To bridge the gap in the R ecosystem and deliver a robust framework for traceable, stateful and secure agent orchestration, this project will leverage the existing functionality of HydraR. In particular, we aim to address this critical need through two objectives:

Objective 1: Memory and State Management

Aim To develop a comprehensive memory and state management system for logging ephemeral agent reasoning and agent interactions.

Rationale Agent memory is often ephemeral and untraceable, making it difficult to capture and audit the entire “journey” of an agentic workflow. Currently, HydraR uses DuckDB to maintain a central memory system that continuously saves workflow checkpoints, ensuring a complete, verifiable record of all AI interactions and data changes. Despite this, **HydraR does not support specialized features required for production-grade bioinformatic pipelines**, such as automated retries for failed tasks, execution caching for reducing redundant computations, and agent output validation for workflow resilience.

Approach To develop a comprehensive memory and state management system, we will introduce several enhancements to HydraR:

- **Restart Checkpointing:** We will implement a system to automatically serialize agents, environment, local variables, and their current reasoning state. Thus, in the event of a failure (e.g., API timeout, resource exhaustion, or logic error), HydraR can reconstitute the exact state of the R session, allowing for automated retries without restarting the entire pipeline.
- **Execution Caching and State Retrieval:** We will implement a content-addressable storage (CAS) model for all agent-generated artifacts. By hashing output files and R objects, and linking them to specific states and reasoning paths, HydraR can maintain a complete audit trail of how each artifact was generated, allowing the system to “skip” previously executed tasks and “resume” from any point in the workflow.
- **Agent Validation and Recovery:** We will implement a system to automatically validate agent outputs. Thus, if the output of an agent fails to validate (e.g., malformed code or incorrect data schema), HydraR can diagnose and rectify the error, ensuring the main research flow continues safely and autonomously.

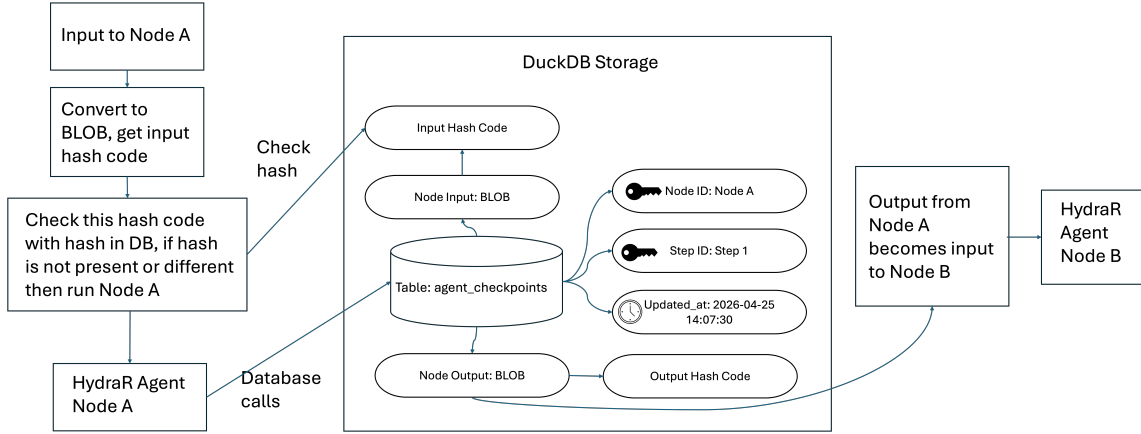


Figure 1: Memory and State Management

Objective 2: Environment Management

Aim To develop a robust environment management system for isolating agents from the host environment.

Rationale Granting autonomous agents unchecked access to sensitive research environments can be dangerous as their actions can inadvertently lead to file modifications, deletions, or corruption of the working directory. Unrestricted access to the working directory also makes it difficult for parallelised agents to work on the same task without overwriting each other's work, which is a major hurdle for scaling agentic workflows. However, **HydraR does not enforce filesystem and runtime isolation and therefore lacks the capabilities to safely and securely orchestrate agentic workflows**, thereby posing a significant barrier to the adoption of agentic systems in R-based research.

Approach To develop a robust environment management system, we will introduce several new features to HydraR:

- **Git Worktrees:** We will implement a Git worktree layer to prevent agents from modifying the filesystem at the same time. By assigning tasks to unique worktrees, HydraR can capture and track all file changes made by the agent, thereby providing an auditable record of every file modification.
- **Docker Integration:** We will implement a Docker layer where each agent execution environment is isolated from the host environment. Every agent can operate in its own ephemeral environment, allowing the orchestrator to spin up and shut down environments as needed. This will also ensure that the agentic workflows are reproducible, as each agent can be executed in the same environment.
- **Singularity and Apptainer for HPC:** We will provide native support for Singularity and Apptainer to accommodate bioinformatic workflows hosted institutional High-Performance Computing (HPC) clusters where Docker may be restricted. However, we will optimize this feature for standard HPC structures (e.g., SLURM integration) to ensure that the agentic infrastructure complies with security and resource policies.

- **Persistent State Tracking via Mounted Worktrees:** To bridge the gap between container volatility and workflow statefulness, we will manage Git worktree logic within persistent Docker mounts. By mounting the worktree directory from the host to the container, HydraR will ensure that the agent's filesystem state is preserved across container restarts. Combined with our restart checkpointing, this will allow an agent to resume from the exact filesystem state it was in before a crash, while maintaining a versioned history of its progress via Git.

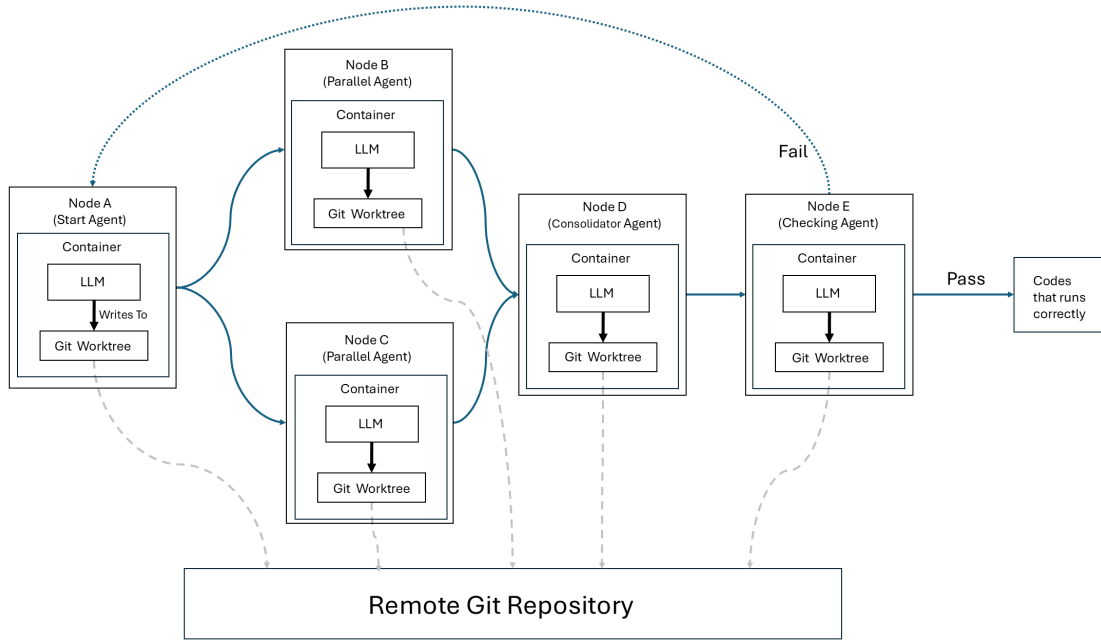


Figure 2: Environment Management

Timeline

Start-up Phase

- **Reporting Framework:** Establish a bi-weekly internal review process to track milestone progress.

Technical Delivery

The proposed work will be delivered over a 7-week period and involve approximately 245 hours of expert developer time. The work is structured into three phases:

- **Phase 1: Memory and State Management (Weeks 1-3):**
 - Development of automated restart checkpointing system.
 - Development of execution caching and state retrieval system.
 - Development of agent validation and recovery system.
- **Phase 2: Environment Management (Weeks 4-6):**
 - Integration with Git worktree.
 - Integration with Docker integration.
 - Compatibility for Singularity/Apptainer.
- **Phase 3: Validation & Community Management (Week 7):**
 - Finalization of Agents and Skills Management tools into lifecycle modules.
 - Documentation, bug fixing, and release processing.

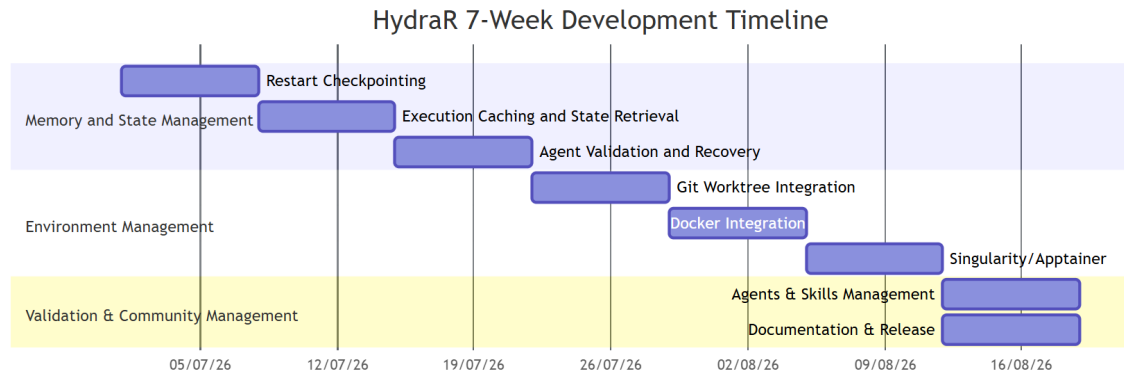


Figure 3: Timeline

Other Aspects

- **Hosting:** Code will be hosted on GitHub under the *APAF-bioinformatics* organization.
- **Conference:** We will attend and present HydraR at relevant conferences to engage with the bioinformatics community.
- **Social media:** We will utilize social media to engage with the bioinformatics community.
- **Publicity:** We will share quarterly updates on the R Consortium blog.
- **Community Engagement:** An early version of the tool is already undergoing the rOpenSci peer-review process and will transition to a community-maintained model.

Budget & Funding Plan

- Total requested budget: **\$14,400 USD**.
- Macquarie University will provide support, covering:
 - **Computational Resources:** Access to High-Performance Computing (HPC) resources for testing and development.

Milestones Budget

Description	R ConsortiumCash (USD)
Milestone 1: Memory and State Management	
Co-CI Tay (1.0 FTE)	\$5,760
Milestone 2: Environment Management	
Co-CI Tay (1.0 FTE)	\$5,760
Milestone 3: Validation & Community Management	
Co-CI Tay (1.0 FTE)	\$2,880
Total	\$14,400

Success

The success of the HydraR infrastructure project will be measured by its adoption, technical stability, and contribution to the R ecosystem's AI readiness.

Success Metrics

- **Package Adoption:** At least 3 existing R/Bioconductor packages demonstrating “Skill Node” integration via the HydraR manifest protocol.
- **Reliability Benchmark:** A published reproducible benchmark dataset showing a >20% improvement in agent task completion rates when using HydraR state management vs. stateless alternatives.
- **Community Engagement:** Successful completion of the rOpenSci peer-review process and transition to a community-maintained model.
- **CRAN/Bioconductor Presence:** Maintain the 0-error/warning/note status across all major R versions.

Definition of Done

The project will be considered successful upon the delivery of:

- A stable CRAN-ready (or Bioconductor-ready) R package named **HydraR**.
- A fully functional memory and state management system that can autonomously retry failed tasks, skip previously executed tasks, or validate agent outputs.
- A fully functional environment memory system that integrates with git worktree, Docker and Singularity/Apptainer.
- Comprehensive documentation, including at least three “reproducible case studies” (vignettes) showcasing traceable agent orchestration.

Measuring success

We will use the following markers to track our progress:

- **Week 2:** Completion of the **duckdb** backend and successful hashing of R objects in a session.

- **Week 4:** Completion of agent validation and integration with git worktree.
- **Week 6:** Completion of integration with docker and compatibility for Singularity/Apptainer
- **Week 7:** Successful community beta-test with at least 5 bioinformatics researchers and final project report and publication of the demonstration blog post on the R Consortium website.

Future work

HydraR is backed by **APAF Bioinformatics** (at Macquarie University), ensuring long-term institutional support and alignment with national-scale bioinformatics infrastructure (NCRIS).

- **Commercial & Institutional Support:** Ongoing maintenance is built into the APAF digital transformation roadmap (2026-2030).
- **Open Source Community:** By providing an interoperable manifest protocol, we encourage a “plugin” ecosystem where community members can contribute drivers and skill nodes without needing deep knowledge of the core DAG engine, ensuring the framework outlives its original funding period.

HydraR is designed to be the foundation for an “Autopoietic Agentic Ecosystem” in R. Future development directions include:

- **Decentralized Orchestration:** Extending the infrastructure to support peer-to-peer agent coordination across distributed R nodes.
- **Advanced Containerization:** Tighter integration with Docker/Singularity for “Secure Execution Enclaves” where agents can test code in isolated environments.
- **Agentic Benchmarking:** A standardized suite for benchmarking LLM performance on bioinformatics-specific R tasks using the HydraR traceability layer.